UT1. Gestión de Acceso a la base de datos

RA 3. Implanta métodos de control de acceso utilizando asistentes, herramientas gráficas y comandos del lenguaje del sistema gestor

Introducción

Una vez instalado y configurado el sistema gestor para poder comenzar a trabajar con él, será necesario habilitar el acceso a los usuarios para que puedan acceder al sistema.

Normalmente los sistemas gestores de bases de datos instalados en entornos de producción son sistemas multiusuario, por lo que es necesario gestionar adecuadamente qué usuarios serán los que podrán acceder al sistema y qué información podrán ver o qué operaciones podrán realizar cada usuario, para garantizar la confidencialidad de los datos almacenados en las bases de datos.

Vistas Personalizadas

- **Vista** es un objeto de base de datos que se comporta como una **tabla virtual**, cuyo contenido se genera dinámicamente mediante una consulta ***SELECT sobre una o varias tablas base.***

- Una **vista personalizada** es aquella diseñada específicamente para **un tipo de usuario o rol**, limitando los datos accesibles y/o adaptando la presentación según las necesidades del usuario

Tipos Vistas Personalizadas



Vistas Estándar



Abstracción y seguridad que permite mostrar solo los datos relevantes de las tablas base, sin almacenar físicamente la información y simplificando la interacción del usuario con la base de datos.

Una vista actualizable es una vista estándar que permite realizar operaciones de modificación **(INSERT, UPDATE o DELETE)** sobre los datos de la tabla subyacente.

Vistas Actualizables





Vistas Materializadas

- ✓ **Almacena físicamente los resultados de una consulta**.
- ✓ **No se recalcula automáticamente cada vez que se consulta**.
- ✓ Se utiliza principalmente para **mejorar el rendimiento** en consultas complejas o con grandes volúmenes de datos.

Tipos Vistas Personalizadas



Cada vista genera una **tabla virtual** que solo se guarda en disco cuando se trata de una vista materializada.

Tipo de Vista	¿Ocupa espacio de datos en disco?	¿Permite modificaciones (INSERT, UPDATE, DELETE)?	¿Se recalculan los datos al consultar?	Casos de uso principales
Vista Estándar	No (0 bytes)	No por defecto (salvo que sea de tipo actualizable)	Sí, en el momento	Ocultar columnas o filas confidenciales y simplificar consultas.
Vista Actualizable	No (los datos se guardan en la tabla base)	Sí (con restricciones: sin agregaciones, GROUP BY, etc.)	Sí, al vuelo sobre la tabla base	Permitir a ciertos usuarios modificar la tabla base respetando reglas de filtrado.
Vista Materializada	Sí (guarda copia física de los datos)	Generalmente no (se actualiza mediante refrescos programados)	No, lee el resultado guardado previamente	Informes complejos y analítica sobre volúmenes gigantescos de datos.

Característica	Vista Estándar / Actualizable	Vista Materializada
Ventajas	• Espacio cero: No malgasta disco duro. • Datos siempre al día: Muestra la información real al milisegundo. • Seguridad: Excelente para ocultar columnas sensibles sin duplicar datos.	• Velocidad extrema: Consultas ultrarrápidas sobre millones de filas. • Ahorro de CPU/RAM: No recalcula agrupaciones o promedios pesados en cada petición.
Desventajas	• Lenta en consultas complejas: Si la vista junta 10 tablas y calcula promedios, la CPU del servidor sufrirá con cada usuario.	• Ocupa disco: Duplica la información físicamente. • Datos "desactualizados": Si la tabla base cambia, la vista no lo refleja hasta que no se ejecute el refresco/sincronización[cite: 3].

Vistas Estándar

Sintaxis para MySQL

```
CREATE [OR REPLACE] VIEW nombre_vista [(columna1, columna2, ...)]
AS
SELECT columnas
FROM tabla_base
WHERE condiciones
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Sintaxis para Oracle

```
CREATE [OR REPLACE] [FORCE | NO FORCE] [EDITIONING | NOEDITIONING] VIEW nombre_vista
[(columna1, columna2, ...)]
AS
SELECT ...
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Recordamos estas 2 reglas:

Los Corchetes [ ] = "Opcional": Puedes incluir lo que hay dentro si necesitas esa función extra, o puedes saltártelo por completo. **¡Nunca se escriben los corchetes [ ] en el terminal real!**

La Barra Vertical | = "Elige solo una opción": Si dentro de un corchete hay varias palabras separadas por |, debes escoger **una única opción** (son incompatibles entre sí).

Vistas Estándar



Sintaxis para MySQL

```
CREATE [OR REPLACE] VIEW nombre_vista [(columna1, columna2, ...)]
AS
SELECT columnas
FROM tabla_base
WHERE condiciones
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Investiga:
Describe la sintaxis anterior:

CREATE [OR REPLACE] view: Crea la vista. Con el nombre de nombre_vista.
Columna1, columna2: Define las columnas de la vista.
SELECT ….. WHERE: Seleccionar las condiciones para generar la vista.
[WITH [CASCADED | LOCAL] CHECK OPTION: Para asegurarse, impide que se inserten o modifiquen datos a través de la vista que no cumplan la condición de la cláusula (Local-> Sobre la vista en la que estás.) CASCADED: Todas las vistas afectadas y encadenadas (se pueden hacer vistas sobre otras vistas).

Vistas Estándar



Sintaxis para Oracle

```
CREATE [OR REPLACE] [FORCE | NO FORCE] [EDITIONING |
NOEDITIONING] VIEW nombre_vista
[(columna1, columna2, ...)]
AS
SELECT ...
[WITH [CASCADED | LOCAL] CHECK OPTION];
```

Investiga:

Describe la sintaxis anterior:
[FORCE | NO FORCE] [EDITIONING | NOEDITIONING].
FORCE -> La creación de la vista, aunque las tablas referenciadas no existan.
NO FORCE -> No se fuerza.
EDITIONING: La vista es sensible a los cambios de las versiones. Si creas una nueva versión del sistema en el futuro, esta vista se podrá versionar de forma independiente sin parar el servidor.
NO EDITIONING: No se pueden hacer nuevas versiones de la vista sin parar el servidor.

Diferencias entre My SQL y Oracle

Característica	MySQL	Oracle
Reemplazo automático: Permite **crear una vista nueva o reemplazar automáticamente** una vista que ya existía	OR REPLACE	OR REPLACE
Crear aunque tabla base no exista ((la vista se crea en estado "inválido" hasta que crees la tabla).	❌ No soporta	✅ FORCE
Edition-Based Redefinition Participe en las diferentes versiones.	❌ No soporta	✅ EDITIONING
Renombrar columnas, nombres diferentes en las vistas y la tabla	✅ Opcional (col1, col2)	✅ Opcional (col1, col2)
Control de integridad, INSERT y UPDATE respeten las condiciones de la vista	✅ WITH CHECK OPTION	✅ WITH CHECK OPTION
Almacenamiento físico	❌ Solo virtual	❌ Solo virtual (materializada es otra cosa)

Ejemplo

Resuelve este ejercicio. No es necesario que lo pruebes en MySQL.

Sobre una tabla de empleados, con la siguientes características:

Columna	Tipo
id	INT
nombre	VARCHAR(50)
departamento	VARCHAR(50)
salario	DECIMAL(10,2)

-> Crear una **vista llamada vista_ventas** que muestre **solo los empleados del departamento "Ventas"**.

-> La vista debe incluir las columnas:

id, nombre y salario.

-> La vista debe garantizar que **cualquier cambio realizado a través de ella cumpla la condición del departamento "Ventas"**.

Vistas Materializadas

```
================================================================================
              CÓDIGO COMPLETO Y EQUIVALENTE (MYSQL vs. ORACLE)
================================================================================

  [ OPCIÓN MYSQL / MARIADB ]                    [ OPCIÓN ORACLE ]
  (Requiere Tabla + Evento)                     (Todo integrado en un solo objeto)

  -- 1. Tabla física para guardar datos         CREATE MATERIALIZED VIEW resumen_dept
  CREATE TABLE resumen_dept AS                  BUILD IMMEDIATE
  SELECT departamento,                          REFRESH COMPLETE
         COUNT(*) AS total_empleados,           NEXT SYSDATE + 1
         AVG(salario) AS salario_promedio       AS
  FROM empleados                                SELECT departamento,
  GROUP BY departamento;                               COUNT(*) AS total_empleados,
                                                       AVG(salario) AS salario_promedio
  -- 2. Evento para refrescar cada día          FROM empleados
  CREATE EVENT actualizar_resumen               GROUP BY departamento;
  ON SCHEDULE EVERY 1 DAY
  DO
    REPLACE INTO resumen_dept
    SELECT departamento, COUNT(*), AVG(salario)
    FROM empleados
    GROUP BY departamento;
================================================================================
```

Vistas Materializadas

Diferencias MySQL vs Oracle

El cálculo (COUNT(*), AVG(salario)):

MySQL: Se escribe en la consulta SELECT al crear la tabla base.

Oracle: Se escribe exactamente igual en la consulta SELECT que va tras la palabra clave AS.

La agrupación (GROUP BY departamento):

MySQL: Va al final de la consulta SELECT de la tabla base.

Oracle: Va exactamente al final del bloque AS SELECT ....

El refresco cada 24 horas:

MySQL: Necesita la orden ON SCHEDULE EVERY 1 DAY dentro de un objeto secundario (CREATE EVENT).

Oracle: Se incluye dentro de la propia vista materializada con la cláusula NEXT SYSDATE + 1.

Vistas Actualizables

Sintaxis para MySQL

```
CREATE VIEW empleados_ventas AS
SELECT id_empleado, nombre, salario
FROM empleados
WHERE departamento = 'Ventas';
```

Sintaxis para Oracle

```
CREATE MATERIALIZED VIEW nombre_vista
[BUILD IMMEDIATE | BUILD DEFERRED]
[REFRESH FAST | REFRESH COMPLETE | REFRESH FORCE]
[START WITH fecha_inicio]
[NEXT fecha_siguiente]
AS
SELECT columnas
FROM tabla_base
WHERE condiciones;
```

Usa las misma sintaxis que las vistas estándar, pero se debe cumplir las condiciones siguientes:

Vistas Actualizables

Condiciones a darse

•Debe basarse en **una sola tabla** o cumplir restricciones si es multi-tabla (cuando en lugar de consultar una sola tabla base, combina datos de dos o más tablas relacionadas mediante sus claves primarias y ajenas.). Cumplir con las restricciones significa que "Una vista multitabla SOLO permite modificar datos si tu orden (INSERT, UPDATE o DELETE) afecta a UNA SOLA TABLA REAL a la vez."

•No debe contener **agregaciones** (SUM, AVG, COUNT) ni GROUP BY.

•No debe tener DISTINCT (que sirve para eliminar filas duplicadas de los resultados que te muestra la pantalla).

•No debe incluir subconsultas complejas en la lista de selección.

•Las columnas modificadas deben **corresponder directamente a columnas de la tabla base**.

Laboratorio en Clase

Vas a crear una base de datos, que va a tener dos tablas, que tendrá la siguiente estructura:

Nombre de tabla	Departamentos
Columnas	id, nombre, ubicacion
Tipos de datos	id → INT (clave primaria) nombre → VARCHAR(50) ubicacion → VARCHAR(50)
Restricciones	id → PRIMARY KEY
Filas de ejemplo	10 registros (Ventas, Marketing, RRHH, IT, Finanzas, Compras, Logística, Legal, Producción, Calidad)

Laboratorio en Clase

Vas a crear una base de datos, que va a tener dos tablas, que tendrá la siguiente estructura:

Nombre de tabla	Empleados
Columnas	id, nombre, departamento_id, salario
Tipos de datos	id → INT (clave primaria) nombre → VARCHAR(50) departamento_id → INT (clave foránea) salario → DECIMAL(10,2)
Restricciones	id → PRIMARY KEY departamento_id → FOREIGN KEY que referencia departamentos(id)
Filas de ejemplo	10 registros con diferentes departamentos y salarios

Laboratorio en Clase

Vas a crear una base de datos, que va a tener dos tablas, que tendrá la siguiente estructura:

Nombre de tabla	Empleados
Columnas	id, nombre, departamento_id, salario
Tipos de datos	id → INT (clave primaria) nombre → VARCHAR(50) departamento_id → INT (clave foránea) salario → DECIMAL(10,2)
Restricciones	id → PRIMARY KEY departamento_id → FOREIGN KEY que referencia departamentos(id)
Filas de ejemplo	10 registros con diferentes departamentos y salarios

- Relación y creación de la tabla

empleados.departamento_id → departamentos.id

Laboratorio en Clase

Crear una **vista estándar** llamada *vista_empleados_altos_salarios* que cumpla las siguientes condiciones:

Incluir las columnas:
- id (empleado)
- nombre (empleado)
- salario
- departamento (nombre del departamento)

Mostrar solo los empleados cuyo salario sea **mayor a 2800**

La vista **no debe permitir insertar o actualizar empleados** fuera de esa condición (opcional: usar WITH CHECK OPTION).

Laboratorio en Clase

Crear una **vista estándar no actualizable** vista_salario_promedio

Condiciones

- Calcular el Salario_promedio
- Las columnas a mostrar son:
  - departamento (nombre del departamento)
  - Salario_promedio.

- Los resultados deben estar ordenados de mayor a menor salario promedio.
- Debes usar: GROUP BY para calcular el salario promedio por departamento

Laboratorio en Clase

Crear una **vista materializada simulada** llamada resumen_salarios:

Condiciones

- departamento_id
- salario_promedio (promedio de salarios de los empleados de ese departamento)

- Debe almacenarse físicamente en una tabla, no solo como consulta virtual.
- Crear un evento programado que actualice la tabla cada 24 horas, simulando el refresco automático de una vista materializada.
- Comparar los tiempos de consulta de la tabla materializada con los de la vista estándar que calcula los salarios promedio directamente.